

Project Idea

Okay — imagine this project like a multiplayer version of a telecom control room game.

You do NOT need telecom knowledge to start.

I'll explain it from absolute beginner level.

Imagine This Scenario

Suppose you and your friends are controlling a city's mobile network.

The city has many mobile towers/cells.

Each cell has:

- * internet traffic
- * users connected
- * speed
- * signal quality
- * latency
- * errors

Suddenly problems happen:

- * too many users connect
- * signal becomes weak
- * latency increases
- * alarms start appearing
- * one tower affects another

Your job:

- * detect the problem
- * choose the correct fix
- * keep the network healthy

That's literally the project.

In Simple Words

This project is basically:

A multiplayer dashboard game where players manage a fake telecom network.

Think of It Like...

Combination of:

Thing Similar To
Multiplayer game Among Us / strategy games
Dashboard Jira / Grafana

Telecom system Mobile network control center
Backend Real enterprise application

What Is O-RAN?

O-RAN is a telecom architecture used in 5G networks.

But for THIS project:
you DO NOT need deep telecom knowledge.

You only need:

- * cells/towers
- * network metrics
- * incidents
- * actions

That's enough.

What Is a "Cell"?

A cell is basically:

- * a mobile tower coverage area.

Example:

Cell-01
Users: 300
Latency: 20ms
Signal: Good
Status: Healthy

What Is an "Incident"?

An incident means:
something bad happened.

Example:

Cell-03 overloaded
Latency high
Packet loss detected

Like:

- * internet slowing down
 - * network issue
 - * tower overload
-

What Does the Player Do?

Player sees the issue and clicks actions.

Example:

Incident	Possible Actions
High latency	Restart cell
Overload	Rebalance traffic
Misconfiguration	Rollback config
Alarm storm	Investigate

Correct action:

- * improves score

Wrong action:

- * damages network

So What Are You ACTUALLY Building?

You are building 4 things together.

1. Frontend (What User Sees)

This is the website/dashboard.

Like the screenshot I showed you.

It displays:

- * network map
- * incidents
- * scores
- * action buttons

Example:

- * Cell-03 becomes red
- * incident appears
- * player clicks "Restart"

This is the UI part.

You'll probably use:

- * React

2. Backend (Main Brain)

This is the Spring Boot application.

It:

- * stores game state

- * handles players
- * calculates score
- * validates actions
- * creates sessions
- * updates incidents

Example:

Player clicked Restart Cell

↓

Backend checks:

Was this correct?

↓

Yes → +50 points

No → -20 points

Backend is basically:
the game engine.

3. Database

Stores:

- * players
- * scores
- * incidents
- * sessions

Probably:

- * MySQL/Postgres

4. Python Simulator

This creates fake network problems automatically.

Example:

Every 10 seconds:

```
if random_number > 7:  
    generate_incident("Cell overload")
```

This makes the game feel alive.

Real-Time Updates

Suppose:
your teammate fixes Cell-03.

Everyone's screen should update instantly.

That's why they want:

- * WebSockets

Like live chat updates.

Why Are Docker & Kubernetes Required?

Because this is also a DevOps project.

They want you to show:

- * deployment skills
- * containerization
- * scaling

Docker

Docker packages app into containers.

Example:

- * frontend container
- * backend container
- * database container

Kubernetes

Kubernetes manages containers.

Example:

- * restart crashed backend
- * scale app
- * manage services

Why SonarQube?

They want clean code.

SonarQube checks:

- * bugs
- * bad code
- * duplication
- * security issues

Why Testing?

Because this is engineering-focused.

You need:

- * unit tests
 - * API tests
 - * integration tests
-

What They REALLY Want to See

Not telecom expertise.

They want to see:

- * software engineering
 - * backend/frontend integration
 - * Docker/K8s
 - * testing
 - * architecture
 - * teamwork
-

Simplest Mental Model

Imagine:

A multiplayer hospital dashboard game
BUT instead of patients,
you manage telecom towers.

That's it.

Your Simplest MVP

You can literally make:

- * 5 fake cells
- * random incidents
- * buttons to fix them
- * score system
- * multiplayer dashboard

And that is enough for a strong submission if done cleanly.

The Hardest Part?

Not coding.

The hardest part is:

- * organizing work
 - * keeping scope small
 - * integrating frontend/backend
 - * debugging real-time updates
-

As Beginners, Your Biggest Advantage

This project is PERFECT for:

- * Spring Boot learning
- * React learning
- * WebSocket learning
- * Docker/K8s practice

Because it naturally combines all of them into one system.

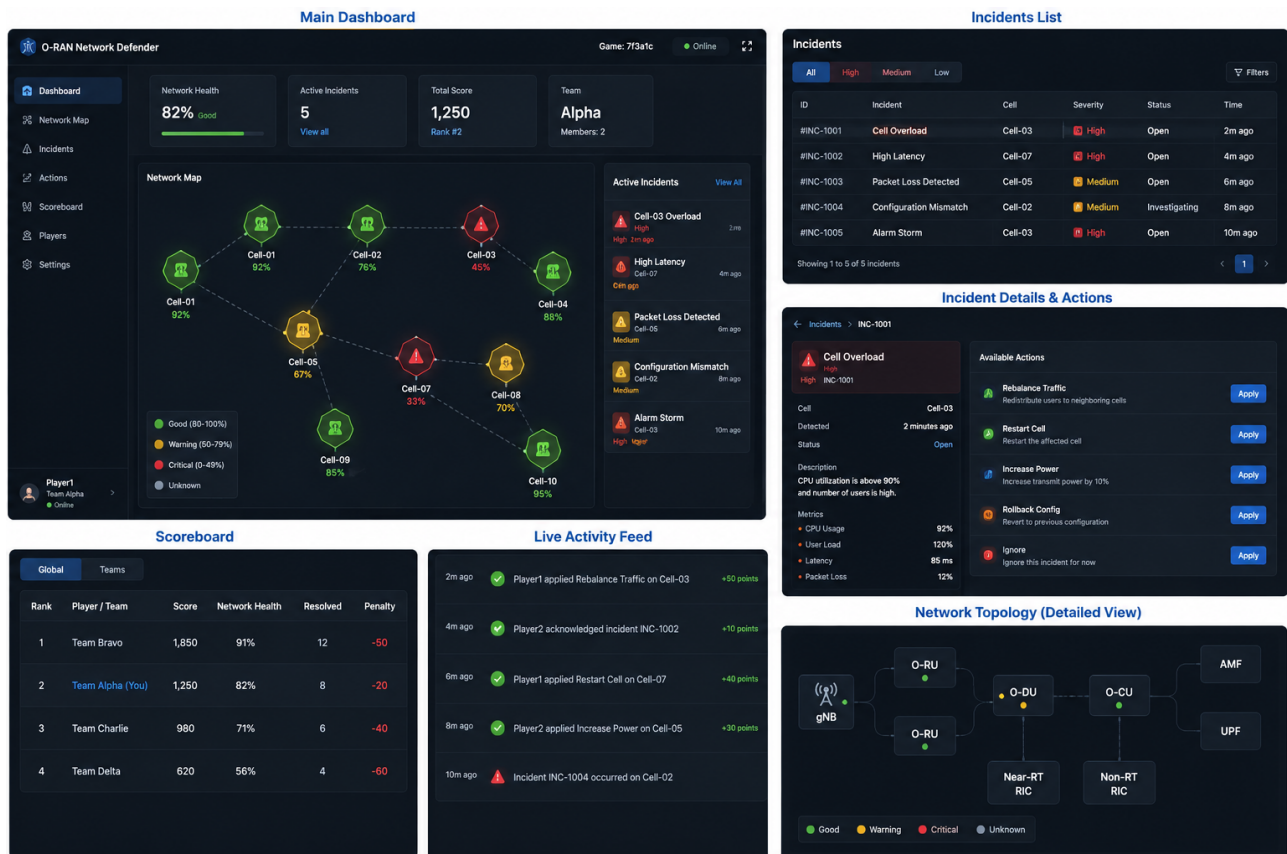


Fig: DashBoard Idea

What is ORAN?

Alright — let's understand O-RAN architecture in the easiest possible way.

Forget the complicated telecom jargon first.

First: What Is RAN?

RAN = Radio Access Network

This is basically:

the part of the mobile network that connects:

- * your phone
to
- * the telecom company network.

Example:

when you use 5G on your phone:

Your Phone



Mobile Tower



Telecom Network



Internet

The tower system is part of the RAN.

Traditional RAN Problem

Earlier telecom systems were:

- * closed
- * proprietary
- * vendor-locked

Meaning:

if a company bought Ericsson equipment,
everything had to be Ericsson.

Very difficult to:

- * mix vendors
- * automate
- * innovate

O-RAN Idea

O-RAN means:

Open Radio Access Network

The idea is:

make telecom networks:

- * modular
- * open
- * programmable
- * intelligent

Like LEGO blocks.

Simplest O-RAN Mental Model

Imagine building a gaming PC.

Instead of:

- * one company making everything,

you can mix:

- * Intel CPU
- * NVIDIA GPU
- * Corsair RAM

O-RAN does the same for telecom networks.

Main O-RAN Components

Now let's understand the architecture.

Big Picture

User Phone



O-RU



O-DU



O-CU



Core Network

And above all of them:

SMO + RIC systems

that manage and optimize the network.

1. O-RU (Radio Unit)

RU = Radio Unit

This is:
the physical radio/tower hardware.

It:

- * sends signals
- * receives signals
- * talks to phones

Closest thing to:
the actual antenna tower.

Example

When you open YouTube:

Phone ↔ Tower Antenna

That antenna area = O-RU.

2. O-DU (Distributed Unit)

DU = Distributed Unit

This handles:

- * real-time processing
- * scheduling
- * radio resource management

Think of it as:
the fast local processing brain.

Easy Analogy

O-RU = ears + mouth
O-DU = fast reflex brain

3. O-CU (Centralized Unit)

CU = Centralized Unit

This handles:

- * higher-level network logic
- * mobility
- * session management

Think:
main control center.

Analogy

O-DU = local office
O-CU = headquarters

4. Core Network

This connects to:

- * internet
- * calls
- * services

Not strictly O-RAN,
but part of full telecom architecture.

Now the IMPORTANT O-RAN Part

O-RAN adds intelligent control systems.

5. RIC (RAN Intelligent Controller)

This is the AI/automation brain.

Two types:

Type	Full Form	Speed
Near-RT RIC	Near Real Time	milliseconds
Non-RT RIC	Non Real Time	seconds/minutes

Near-RT RIC

Makes quick decisions.

Example:

- * congestion detected
- * rebalance traffic immediately

Very fast optimization.

Non-RT RIC

Makes slower strategic decisions.

Example:

- * analyze weekly traffic
 - * optimize policies
 - * train ML models
-

6. xApps and rApps

These are small apps/plugins running inside RIC.

Like mobile apps for network optimization.

Example xApps

- * load balancer
 - * interference detector
 - * power optimizer
-

7. SMO (Service Management & Orchestration)

This is the TOP management layer.

SMO controls:

- * monitoring
- * automation
- * orchestration
- * configurations
- * deployments

Think:
central admin dashboard.

Easy Analogy for Entire Architecture

SMO = CEO
RIC = smart managers
O-CU = headquarters
O-DU = local branches
O-RU = workers/towers

How This Relates To YOUR PROJECT

Your project is NOT building real telecom software.

You are SIMULATING these ideas.

Example in Your Game

Suppose:

Cell-03 overloaded

Your game can simulate:

Component	Role in Game
O-RU	cell/tower
O-DU	manages nearby cell processing
O-CU	central region manager
RIC	auto optimization engine
SMO	dashboard/control system

Example Incident Flow

Cell overloaded
↓
Alarm generated
↓
RIC suggests traffic rebalance
↓
Player approves action
↓
Network health improves
↓
Score increases

That is basically your gameplay loop.

Why Telecom Companies Love O-RAN

Because:

- * open standards
 - * cheaper
 - * multi-vendor
 - * programmable
 - * AI-friendly
 - * scalable
-

Real-Life Companies Using O-RAN

Examples:

- * Ericsson
 - * Nokia
 - * Samsung
 - * Rakuten Mobile
 - * Vodafone
-

MOST IMPORTANT THING

For your project:
you do NOT need deep telecom expertise.

You only need:

- * cells
- * incidents
- * metrics
- * actions
- * scoring
- * some O-RAN terminology

That's enough to make the project believable and technically correct.

Database design idea

Yes. For 1-week MVP, use this database design:

users

- id (PK)
- username
- role
- created_at

game_sessions

- id (PK)
- session_code
- name
- status // WAITING, ACTIVE, ENDED
- started_at
- ended_at
- created_by_user_id (FK users.id)

players

- id (PK)
- user_id (FK users.id)
- game_session_id (FK game_sessions.id)
- team_name
- score
- joined_at

network_cells

- id (PK)
- game_session_id (FK game_sessions.id)
- cell_name
- signal_quality
- user_load
- latency
- packet_loss
- alarm_count
- energy_usage
- health_status // GOOD, WARNING, CRITICAL

incidents

- id (PK)
- game_session_id (FK game_sessions.id)
- cell_id (FK network_cells.id)
- incident_type
- severity // LOW, MEDIUM, HIGH
- status // OPEN, RESOLVED, FAILED
- description
- root_cause
- created_at
- resolved_at

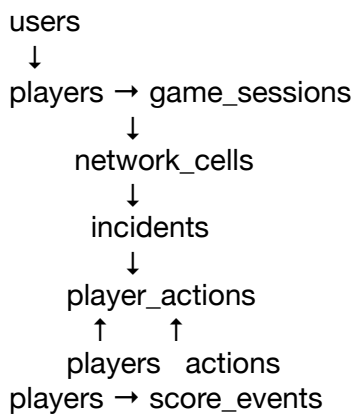
actions

- id (PK)
- action_name
- description

player_actions
- id (PK)
- player_id (FK players.id)
- incident_id (FK incidents.id)
- action_id (FK actions.id)
- result // SUCCESS, FAILED, PARTIAL
- points_awarded
- submitted_at

score_events
- id (PK)
- player_id (FK players.id)
- game_session_id (FK game_sessions.id)
- reason
- points
- created_at

Simple Relationship Diagram



Why this design is enough

This covers the main project requirements:

- * users can join a game
- * multiple players can be in one session
- * each session has network cells
- * cells can have incidents
- * players can submit actions
- * actions affect score
- * score history is stored
- * game state can be shown on dashboard

Example Flow

User joins game



Player record is created



Game has 10 network cells



Python simulator creates incident for Cell-03



Player selects "Rebalance Traffic"



Backend checks result



player_actions entry is saved



score_events entry is saved



player score updates

For your MVP, this database is strong enough. Don't add more tables unless you finish early.